# Shadow-AI Radar: Project Summary

*A local, dependency-free tool that finds unsanctioned AI usage and sensitive-data leakage in a company's web-proxy logs.*

---

## 1. The problem

Employees paste source code, customer data and credentials into consumer AI tools (ChatGPT, Gemini and others) that the company never vetted. Most security teams cannot see it. Shadow-AI Radar reads egress-proxy logs and answers four questions: **who uses which AI tools, which of those are unapproved, what sensitive data went into them, and what should the security team do next?**

Intended users: a security / GRC team at a company, working alongside an HR, Legal and privacy function.

## 2. How it works

```
proxy logs (Squid / CSV / JSONL)
   → convert    common schema; URLs with spaces handled; error lines skipped
   → ingest     parallel, incremental, crash-safe; redact-before-store; opt. pseudonymization
   → enrich     AI-tool catalog + unknown-AI discovery; secret/PII scan; sensitivity classifier
   → store      SQLite: events, user-day, service, department, week
   → detect     5 alert types, correlated and de-duplicated
   → act        dashboard · CLI triage · blocklist · SIEM JSONL · CSV · notifications
```

Alerts: **SECRET_LEAK** (keys, tokens, passwords, private keys), **PII_LEAK** (SSN, Luhn-valid cards), **EXFIL_VOLUME** (unusually large uploads vs. department peers), **SHADOW_ADOPTION** (an unvetted tool spreading across users), **OFF_HOURS** (activity at hours unusual for that user).

## 3. The four disciplines

| Discipline | What was built |
|---|---|
| **Data engineering** | Bronze → silver → gold pipeline on SQLite; process-pool parallelism; byte-offset checkpoints so re-runs read only new data; one transaction per 5,000-line chunk; data-quality rejects (malformed, duplicate, bad timestamps); retention; log adapters |
| **AI engineering** | Prompt-sensitivity classifier (public / internal / confidential / source code) with labelling → training → evaluation loop (active learning, held-out sets, per-class precision/recall); optional AI triage summaries with an offline fallback; classifier down-weighted until trained on real data |
| **Data analysis** | Robust median/MAD volume anomalies vs. department peers; per-user baselines for off-hours; adoption tracking; risk scoring (tool policy × data sensitivity); week-over-week trend normalised per active day |

| Discipline | What was built |
|---|---|
| **Cybersecurity** | Secret/PII detection (patterns, Luhn, entropy, context rules); redact-before-store; ReDoS-hardened parsing; tamper-evident audit log; authenticated dashboard; encrypted backups; threat model |

## At a glance

| | |
|---|---|
| Code | ~2,100 lines across 21 modules; zero third-party dependencies (Python 3.10+) |
| Speed (8 cores) | ≈ 16–21k events/s; 255k events ingested in ≈ 12–15 s; re-run with no new data 0.06 s |
| Interface | 23 CLI commands, a localhost dashboard, JSON API |
| Docs | README, ROADMAP, and `docs/`: PRIVACY, SECURITY, COMPLIANCE, PILOT, and this summary (Markdown, PDF, Word) |

## 4. Capability inventory

- **Operate:** policy-as-config, alert lifecycle (open / ack / resolved / false positive, notes, auto-reopen on recurrence), dashboard with analyst/viewer roles, watch mode, systemd/cron generation (generates only, installs nothing), desktop notifications, analyst training mode (8 hidden cases with an answer key).
- **Integrate:** Squid and generic CSV converters, HR/IdP join (managers on alerts), proxy blocklist, SIEM-friendly JSONL, audit CSV with hashes.
- **Protect (the tool itself):** named users, constant-time token checks, lockout, Host/Origin checks, CSP and other headers, owner-only files (`umask 077`), hash-chained audit log with off-box anchors, AES-256 encrypted backups verified on restore, log-injection and CSV-formula-injection guards.
- **Comply:** keyed pseudonymization of user IDs, data-subject export and erasure, automatic 90-day retention, `radar doctor` posture check, `radar evidence` auditor bundle, GDPR / SOC 2 / ISO 27001 control mapping with an explicit gap list.

## 5. Adoption checklist

| # | Step | Owner |
|---|---|---|
| 1 | Pilot on one department's real proxy logs in shadow mode (`docs/PILOT.md`) | Security |
| 2 | Label ≈ 500 prompts, retrain, reach ≥ 90% accuracy and ≥ 90% confidential recall on a held-out set | Security / data |
| 3 | Complete and sign `docs/PRIVACY.md`; establish lawful basis; notify employees (works council where required) | Legal / HR / DPO |
| 4 | Encrypt the host disk | IT |
| 5 | Independent penetration test | Security |

| # | Step | Owner |
|---|---|---|
| 6 | Install the generated schedule; store audit anchors off-box | IT / Security |

`radar doctor` reports on items 3 to 5, and on a missing audit anchor, until they are done.

## 6. Quick start

```
python3 -m radar run --rebuild                 # demo: synthetic data → out/report.html
python3 -m radar training start                # practice triage with a hidden answer key
RADAR_USERS="alice=analyst:<16+ char token>" \
  python3 -m radar serve                       # dashboard (refuses to start without logins)
python3 -m radar convert squid access.log out/proxy.jsonl --identity hr.csv
python3 -m radar doctor                        # security / privacy posture
python3 -m unittest discover tests             # run the test suite
python3 tools/build_docs.py                    # rebuild this summary as PDF and Word
```

## 7. Repository map

```
radar/    21 modules: pipeline, detectors, catalog, model, analytics, adapters, serve, audit,
          privacy, compliance, backup, export, notify, schedule, training, labeling, triage,
          report, generate, corpus, __main__
tests/    7 test files + _cleanup.py        eval/    held-out classifier sets (synthetic)
config/   policy.json                       bench/   throughput benchmark
docs/     PRIVACY · SECURITY · COMPLIANCE · PILOT · PROJECT_SUMMARY
tools/    build_docs · md2html · md2docx · verify_docx   (Markdown → PDF / Word)
```

## 8. What makes it distinctive

- It spans **data engineering, AI engineering, data analysis and cybersecurity** in one product aimed at a current corporate risk: shadow AI.
- It is **privacy-first by construction:** secrets and PII are redacted before storage, identities can be pseudonymized, and subject rights and retention are built in.
- It **runs anywhere** with only the Python standard library.